



Security Audit Report

USDR Redemption V2

V2 · FIX VERIFICATION

INITIAL AUDIT	45611d4
FIX REVIEW	ac06719
DATE	2026-06-19

CIMORA

CONTENTS

Executive Summary	2
L-01 · <code>maxRedeemableUSDR</code> reverts with <code>ZeroPayout</code> at 1-unit USDC	3
G-01 · Drop redundant <code>balanceOf</code> check in <code>_redeem</code>	5
G-02 · Wrap <code>lastFundingTime + SWEEP_DELAY</code> in <code>unchecked</code>	7
G-03 · Use <code>ReentrancyGuardTransient</code> for ~4,800 gas savings	9
Observations	10
Appendix · Methodology	43
Appendix · Severity Scale	43
Appendix · Fix Status Scale	44
Appendix · Scope	44
Appendix · Disclaimer	44
Appendix · Related-Party Disclosure	45

PROJECT	USDR Redemption V2
DATE	2026-06-19
INITIAL AUDIT	45611d4
FIX REVIEW	ac06719
AUDITOR	Cimora

Executive Summary

The engagement focused on one critical spec-correctness issue and three gas optimizations in the redemption flow. L-01 was the central concern: `maxRedeemableUSDR()` violated its documented guarantee by returning amounts that would revert with `ZeroPayout` at dust balances. The fix implements the spec recommendation verbatim—checking `previewRedeem()` and returning 0 when payout is zero; tests confirm the 1-unit dust case now returns 0 and redeemable amounts appear once payout ≥ 1 .

Gas optimizations landed cleanly. G-02 wrapped `lastFundingTime + SWEEP_DELAY` in unchecked blocks at both call sites (sweep and sweepUnlockTime) with overflow-safe comments; G-03 upgraded from storage `ReentrancyGuard` to transient `ReentrancyGuardTransient` with Cancun EVM targeting for ~4,800 gas per guarded call. One flag remains by-design: G-01 identified the `balanceOf` pre-check in `_redeem()` as redundant, but the team retained it to preserve the bespoke `InsufficientUSDC` error for integrator UX. This is a deliberate gas-versus-UX trade-off. No residual correctness concerns remain.

This is the **v2 fix-review report** for USDR Redemption V2, covering the verification of fixes applied to the findings raised in the initial audit. The initial audit was performed at commit `45611d4`; fixes were re-verified at commit `ac06719`. Of the 4 findings carried forward from the initial audit, 3 resolved. Each finding below carries a fix-status badge (Resolved / Partial / Mitigated / Unresolved / New) and a fix-verification callout describing what the auditor observed at the fix-review commit.

SEVERITY OVERVIEW



Low 1 Gas 3

FIX STATUS



Resolved 3 Unresolved 1

L-01 · maxRedeemableUSDR reverts with ZeroPayout at 1-unit USDC

LOW

FINDING 1 OF 4

L-01

LOCATION src/USDRRedemption.sol:L198-L213

FIX STATUS Resolved

FIX VERIFICATION · RESOLVED

src/USDRRedemption.sol:213-216 now implements the recommended minimal fix verbatim: maxRedeemableUSDR() computes $m = (\text{availableUSDC}() * \text{USDR_UNIT}) / \text{rate}$, then if $(\text{previewRedeem}(m) == 0)$ return 0;, so it never advertises an amount that would revert with ZeroPayout. The original 1-unit dust example ($m=1846$, $\text{previewRedeem}=0$) now returns 0, closing the spec-R3 violation; test_maxRedeemable_returnsZeroAtDustBalance (test/USDRRedemption.t.sol:574-589) and testFuzz_maxRedeemableUSDR_neverReverts (591-603) confirm the helper returns 0 at 1 raw unit and a redeemable amount once payout ≥ 1 . Although the client's stated position is 'acknowledged', the code change actually resolves the root cause, so the correct verdict is resolved rather than acknowledged.

maxRedeemableUSDR() violates its documented guarantee. Per spec R3, the helper should return the largest USDR amount that redeems without reverting and always pays out within available balance; at 1 raw unit of USDC it returns an amount that reverts instead. The cause is inconsistent rounding: inverse rounding (round down) in maxRedeemableUSDR() diverges from forward rounding (round down) plus the ZeroPayout guard in _redeem().

SOLIDITY

```
// line 198-200
function previewRedeem(uint256 usdrAmount) public view returns (uint256) {
    return (usdrAmount * rate) / USDR_UNIT;
}

// line 211-213
function maxRedeemableUSDR() external view returns (uint256) {
    return (availableUSDC() * USDR_UNIT) / rate;
}

// line 140-142
function _redeem(uint256 usdrAmount, address receiver) internal nonReentrant returns (uint256 usdcAmount) {
    usdcAmount = previewRedeem(usdrAmount);
    if (usdcAmount == 0) revert ZeroPayout();
}
```

Concrete example (rate = 541700, USDR_UNIT = 1e9):

- availableUSDC() = 1 (0.000001 USDC)
- maxRedeemableUSDR() = $\text{floor}(1 * 1e9 / 541700) = \text{floor}(1846.04) = 1846$
- Caller invokes redeem(1846)
- previewRedeem(1846) = $\text{floor}(1846 * 541700 / 1e9) = \text{floor}(0.9999782) = 0$
- _redeem reverts with ZeroPayout()

The failure occurs when $\text{floor}(\text{available} * 1e9 / \text{rate}) * \text{rate} < 1e9$. For rate = 541700, this happens only at available = 1; for available ≥ 2 , the payout is guaranteed ≥ 1 USDC. A 1-unit residual is typical after redemptions draw the balance down.

IMPACT

A view helper for frontends. A caller trusting the returned amount will invoke a redeem that reverts, wasting gas and confusing the user — but no fund loss. The `redeem` path itself is correct; it properly reverts rather than burning USDR for zero payout. The bug is purely inconsistency between the helper's promise and the redeem path's rounding plus guard.

REMEDIATION

Make the helper agree with the redeem path's effective minimum. Either (a) return 0 when the resulting amount would preview to zero payout, or (b) clamp/round the returned amount up to the next value whose `previewRedeem` is ≥ 1 . Minimal fix:

```
function maxRedeemableUSDR() external view returns (uint256 m) {
    m = (availableUSDC() * USDR_UNIT) / rate;
    if (previewRedeem(m) == 0) return 0; // never advertise an amount that would revert
}
```

SOLIDITY

Returning 0 honestly communicates that no non-reverting redemption is possible at that dust balance.

CLIENT RESPONSE · ACKNOWLEDGED

The client confirmed the issue in `maxRedeemableUSDR()` violating spec R3 and has corrected it in commit `a4e4a01`. The fix implements the suggested remediation: the function now returns 0 when the inverse-rounded amount would preview to zero payout, preventing it from advertising amounts that would revert with `ZeroPayout`. The implementation is supported by `test_maxRedeemable_returnsZeroAtDustBalance`, which validates the 1-row-unit dust-balance edge case and recovery at 2 units.

G-01 · Drop redundant balanceOf check in _redeem

GAS

FINDING 2 OF 4

G-01

LOCATION src/USDRRedemption.sol:L140-L153

FIX STATUS Acknowledged

FIX VERIFICATION · ACKNOWLEDGED

At src/USDRRedemption.sol:127-128 the explicit pre-check remains: `uint256 available = availableUSDC(); if (usdcAmount > available) revert InsufficientUSDC(usdcAmount, available);` — where `availableUSDC()` (L200-202) is the `usdc.balanceOf(address(this))` STATICCALL the finding flagged as redundant. The check was deliberately retained, not removed. The client's by_design rationale is sound: this is a gas-severity optimization, and keeping the bespoke `InsufficientUSDC(required, available)` error genuinely yields better integrator UX than the generic `ERC20 revert`, which is materially relevant under the contract's FCFS model where underfunded redeems are a routine race outcome. The invariant I1 (full payout or revert) holds either way, so accepting the measured ~1k gas cost for a richer revert is a reasonable, well-reasoned trade-off rather than an unintended inefficiency.

In `_redeem`, the contract makes an explicit external `balanceOf` STATICCALL purely to pre-validate that it can cover the payout:

```
function _redeem(uint256 usdrAmount, address receiver) internal nonReentrant returns (uint256 usdcAmount) {
    usdcAmount = previewRedeem(usdrAmount);
    if (usdcAmount == 0) revert ZeroPayout();

    uint256 available = usdc.balanceOf(address(this)); // <-- extra external call
    if (usdcAmount > available) revert InsufficientUSDC(usdcAmount, available);

    usdr.burn(msg.sender, usdrAmount);
    usdc.safeTransfer(receiver, usdcAmount);

    emit Redeemed(msg.sender, receiver, usdrAmount, usdcAmount);
}
```

SOLIDITY

The subsequent `usdc.safeTransfer(receiver, usdcAmount)` already reverts the entire transaction (rolling back the `usdr.burn`) whenever the contract holds less than `usdcAmount` USDC. The pre-check is therefore functionally redundant for invariant I1 ('never transfers out more USDC than held; full payout or revert') — that invariant still holds via the transfer reverting.

The redundant `balanceOf` STATICCALL costs roughly ~2400-2600 gas on every successful redeem: the STATICCALL itself plus an SLOAD of the balance mapping. (The cold→warm account-access savings mostly shift to the transfer, but the entire `balanceOf` execution is eliminated.)

Removing it:

SOLIDITY

```
function _redeem(uint256 usdrAmount, address receiver) internal nonReentrant returns (uint256 usdcAmount) {
    usdcAmount = previewRedeem(usdrAmount);
    if (usdcAmount == 0) revert ZeroPayout();

    // Burn USDR straight from the redeemer (allowance-based; never custodied here),
    // then pay USDC; safeTransfer reverts (rolling back the burn) if the balance
    // cannot cover the full payout, preserving the all-or-nothing invariant.
    usdr.burn(msg.sender, usdrAmount);
    usdc.safeTransfer(receiver, usdcAmount);

    emit Redeemed(msg.sender, receiver, usdrAmount, usdcAmount);
}
```

Trade-off: This removes the bespoke `InsufficientUSDC(required, available)` custom error. On an underfunded redeem the call would instead revert with USDC's own transfer error (e.g., `ERC20InsufficientBalance`), which is less informative for UIs that surface the available balance. The `InsufficientUSDC` error is an explicit design choice in this contract, and `maxRedeemableUSDR()` / `availableUSDC()` already let a UI size a redeem off-chain before sending.

If the richer revert is worth ~2.5k gas per redeem, keep the check; otherwise this is a clean, behavior-equivalent-for-I1 saving on the single most frequently called function.

Gas verification: Compiled under the project's Foundry profile (solc 0.8.30, optimizer on, runs 1000, via_ir false); measured ~980 gas per successful `redeem()` call (40895 → 39915 in `test_redeem_explicitReceiver`), bytecode shrank 207 bytes (3854 → 3647), via forge test. The eliminated `STATICCALL+SLOAD` is cheaper than the ~2.5k estimate because the USDC address is already warm in the test path.

CLIENT RESPONSE · BY DESIGN

The client confirms that the redundant `balanceOf` pre-check in `_redeem` (L140) is intentional, not an oversight. The contract retains the check specifically to produce the custom `InsufficientUSDC(required, available)` error, which surfaces both the requested payout amount and the current available balance to integrators. Under the contract's FCFS (first-come-first-served) redemption model, underfunded redeems are an expected and common outcome when the pot is drained between a UI's read and the user's transaction; the client argues that providing both required and available values is materially better UX than the generic `ERC20InsufficientBalance` revert that `safeTransfer` alone would produce. The client explicitly accepts the ~1k measured gas cost per successful redeem as a deliberate design trade-off, framing error richness as an architectural goal rather than acknowledging the cost of an unintended inefficiency. Off-chain sizing functions (`maxRedeemableUSDR()`, `availableUSDC()`) are available to integrators seeking to avoid underfunded transactions.

G-02 · Wrap lastFundingTime + SWEEP_DELAY in unchecked

GAS

FINDING 3 OF 4

G-02

LOCATION src/USDRRedemption.sol:L174

FIX STATUS Resolved

FIX VERIFICATION · RESOLVED

Both call sites now wrap the addition in unchecked: sweep() at src/USDRRedemption.sol:165-167 (unlockTime = lastFundingTime + SWEEP_DELAY;) and sweepUnlockTime() at src/USDRRedemption.sol:222-224 (return lastFundingTime + SWEEP_DELAY;), each with an explanatory comment noting the sum cannot overflow uint256. This matches the recommended remediation exactly. The client's acknowledged rationale is sound and the fix is genuinely present in this commit, so the issue is resolved.

The expression lastFundingTime + SWEEP_DELAY appears in sweep() (line 174) and sweepUnlockTime() (line 217). lastFundingTime is a block-timestamp value (always block.timestamp, bounded by 2^{32} for centuries) and SWEEP_DELAY is a constant 180 days (1.5e7). Their sum cannot overflow uint256, so the overflow guard that solc 0.8.x inserts (a duplicated ADD plus overflow check and revert branch) is dead weight. The optimizer doesn't remove it because it cannot statically prove lastFundingTime is bounded; it's a mutable storage value. Wrapping the addition in unchecked removes the guard, shrinks bytecode, and saves a small amount of gas per call.

sweep() is owner-only and rare; sweepUnlockTime() is a view (free off-chain). The main beneficiary is any on-chain integrator reading sweepUnlockTime(). Estimated savings: ~20-40 gas per call; correctness unchanged since overflow is impossible.

Unoptimized (src/USDRRedemption.sol:172-179):

```
function sweep(address to) external onlyOwner {
    if (to == address(0)) revert ZeroAddress();
    uint256 unlockTime = lastFundingTime + SWEEP_DELAY;
    if (block.timestamp < unlockTime) revert SweepLocked(unlockTime);
    uint256 balance = usdc.balanceOf(address(this));
    usdc.safeTransfer(to, balance);
    emit Swept(to, balance);
}
```

SOLIDITY

Optimized:

```
function sweep(address to) external onlyOwner {
    if (to == address(0)) revert ZeroAddress();
    uint256 unlockTime;
    unchecked {
        unlockTime = lastFundingTime + SWEEP_DELAY;
    }
    if (block.timestamp < unlockTime) revert SweepLocked(unlockTime);
    uint256 balance = usdc.balanceOf(address(this));
```

SOLIDITY


```

    usdc.safeTransfer(to, balance);
    emit Swept(to, balance);
}

```

Unoptimized (src/USDRRedemption.sol:216-218):

```

function sweepUnlockTime() external view returns (uint256) {
    return lastFundingTime + SWEEP_DELAY;
}

```

SOLIDITY

Optimized:

```

function sweepUnlockTime() external view returns (uint256) {
    unchecked {
        return lastFundingTime + SWEEP_DELAY;
    }
}

```

SOLIDITY

Gas measurements (solc 0.8.30, optimizer on, runs 1000, via_ir false):

- `sweepUnlockTime()` : 2449 → 2344 gas (~105 per call)
- `sweep()` : median 26247 → 26168, max 56622 → 56543 (~79 on paths executing the addition; zero-address revert path skips it)
- Deployed runtime code: -56 bytes (3854 → 3798)

Wrap `lastFundingTime + SWEEP_DELAY` in `unchecked` at both call sites (lines 174 and 217). The overflow check is genuinely dead weight; the optimizer does not remove it without `via_ir`.

CLIENT RESPONSE · ACKNOWLEDGED

The client confirms the fix has been implemented in commit fc0b82c, wrapping `lastFundingTime + SWEEP_DELAY` in `unchecked` blocks at both call sites (`sweep()` at line 174 and `sweepUnlockTime()` at line 217). They demonstrate understanding of the underlying rationale: a block timestamp value (bounded to $\sim 2^{32}$ for centuries) plus a constant 180-day delay cannot overflow `uint256`, so the solc 0.8.x overflow guard is dead weight. The implementation matches the auditor's recommended approach and should yield the estimated 20–40 gas savings per call on `sweepUnlockTime()` and ~79 gas on the `sweep()` path that executes the addition, plus ~56 bytes of bytecode reduction.

G-03 · Use ReentrancyGuardTransient for ~4,800 gas savings

GAS

FINDING 4 OF 4

G-03

LOCATION src/USDRRedemption.sol:L8-L43

FIX STATUS Resolved

FIX VERIFICATION · RESOLVED

The recommendation was implemented exactly: src/USDRRedemption.sol:9 now imports ReentrancyGuardTransient and line 48 inherits from it instead of the storage-based ReentrancyGuard. The nonReentrant modifier on _redeem (line 123) and other functions resolves to the transient (EIP-1153 TSTORE/TLOAD) guard, and foundry.toml pins solc_version=0.8.30 with evm_version=cancun (line 9), so the Cancun opcodes are emitted for the Polygon PoS target. The client's stated rationale is sound and matches the code; the gas optimization is fully realized.

_redeem, the contract's primary user-facing function, is guarded by OpenZeppelin's storage-based ReentrancyGuard :

```
// line 8
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
...
// line 43
contract USDRRedemption is Ownable2Step, ReentrancyGuard {
...
// line 140
function _redeem(uint256 usdrAmount, address receiver) internal nonReentrant returns (uint256 usdcAmount) {
```

SOLIDITY

This guard stores its _status flag in regular storage. Each protected call burns ~5,000 gas on entry (cold SSTORE: 2,100 base + 2,900 reset) plus ~2,900 on exit (reset back to NOT_ENTERED), for a net unrefundable cost around 5,000 gas per redeem.

OpenZeppelin 5.6.1 ships a drop-in replacement, ReentrancyGuardTransient, which stores the flag in transient memory (EIP-1153 TSTORE / TLOAD, 100 gas each). The modifier name is unchanged, so the swap is mechanical.

Your foundry.toml pins solc_version = "0.8.30" with no evm_version override, targeting Cancun where TSTORE / TLOAD are native. Polygon PoS has supported EIP-1153 since Napoli. The guard emits different bytecode — TSTORE / TLOAD instead of SSTORE / SLOAD — so the saving is real, not an optimization artifact.

Measured impact (forge test -gas-report): median -1,905 gas, min -4,765, avg -1,925. Deployed bytecode grows by 21 bytes.

REMEDIATION

```
// line 8
import {ReentrancyGuardTransient} from "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";
...
// line 43
contract USDRRedemption is Ownable2Step, ReentrancyGuardTransient {
```

SOLIDITY

No other changes needed; `_redeem`'s `nonReentrant` modifier resolves to the transient version. The transient guard requires EIP-1153, so keep this only if your deployment target supports Cancun opcodes.

CLIENT RESPONSE · ACKNOWLEDGED

Client has implemented the `ReentrancyGuardTransient` optimization as recommended. They switched from the storage-based `ReentrancyGuard` to `ReentrancyGuardTransient` (EIP-1153) in commit 99a624c and confirmed the fix is active by verifying that `TSTORE/TLOAD` opcodes appear in the deployed bytecode. They also pinned `evm_version=cancun` in `foundry.toml`; the toolchain default had previously resolved to a post-Cancun version, but Cancun is the appropriate baseline for Polygon PoS deployment. The gas optimization has been fully realized.

Observations

The following items were considered during the audit but did not meet the bar for inclusion in the formal findings list (insufficient impact, defense-in-depth-only, internal documentation, or theoretical paths). They are surfaced here as housekeeping the development team may choose to address outside the formal remediation flow.

O-01 · Owner can extract USDC backing via timelocked sweep

LOCATION `src/USDRRedemption.sol:L172-L179`

Document prominently in user-facing UI that USDC redemption backing is owner-recoverable via sweep on a 180-day timelock and redemption depends on owner fund calls.

FIX VERIFICATION · ACKNOWLEDGED

The finding requests off-chain/UI documentation, not a code change — the mechanism is real and intentional. In the new revision the sweep timelock is implemented and documented exactly as described: sweep (`src/USDRRedemption.sol:160-172`) gates withdrawal of the full `availableUSDC()` balance behind `lastFundingTime + SWEEP_DELAY` (180 days), and `NatSpec` at lines 157-159, 162-167, and 174-179 spells out the 180-day timelock, its reset on each `fund()` call (lines 145-153, invariant I5), and that USDC is explicitly excluded from `rescueERC20` so it can never bypass the timelock. The client's rationale is sound: the on-chain behavior and its safeguards are accurately documented in the code, and UI/user-facing communication is legitimately an off-chain task warranting no contract change.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges that the mechanism is real and intentional—the owner can recover USDC backing via sweep on a 180-day timelock that resets on each `fund()` call (invariant I5)—and confirms it is already documented in the contract's `NatSpec`. The finding's request for prominent user-facing UI communication of this feature is accepted as an off-chain documentation task tracked in the project's issue management system outside the code repository. No contract code change is warranted, as the technical behavior and its safeguards are already documented in the code.

O-02 · fund() is a MEV target that a single bot can drain

LOCATION src/USDRRedemption.sol:L140-L153

Consider submitting fund() via a private relay (e.g. Flashbots) to reduce the public backrun signal on the FCFS pool; intended no-per-user-cap design, no protocol loss.

FIX VERIFICATION · ACKNOWLEDGED

fund() at src/USDRRedemption.sol:145-154 is unchanged and remains an owner-only top-up into a shared FCFS pool with no per-user cap — consistent with the acknowledged design. The finding involves no protocol loss (only MEV/fairness on a public pool), and the client's operational mitigation (submitting fund() via a Flashbots private relay to suppress the public backrun signal) is a sound, concrete remedy for a risk that cannot be eliminated in-contract without abandoning the intentional no-per-user-cap design. I concur with the rationale.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the backrunning MEV risk on fund() top-ups at L140 and confirms the no-per-user-cap FCFS design is intentional, requiring no contract changes. The mitigation is operational: the funding multisig will submit fund() calls via Flashbots private relay to suppress the public backrun signal and reduce bot extraction. This is a concrete off-chain practice that addresses the finding's recommendation without code changes.

O-03 · Reentrancy guard on redeem is untested

LOCATION `src/USDRRedemption.sol:L140-L153`

Add a reentrancy unit test for `_redeem` using a malicious mock USDC whose transfer re-enters `redeem`, asserting revert with `ReentrancyGuard.ReentrancyGuardReentrantCall.selector`.

FIX VERIFICATION · RESOLVED

`test/mocks/ReentrantUSDC.sol` implements a 6-decimal malicious USDC whose `transfer()` re-enters `target.redeem(1)` during the payout (lines 32-39), and `test/USDRRedemption.t.sol:327-346` (`test_redeem_reentrancyGuarded`) wires it up, sets the attack flag, and asserts the redeem reverts with `ReentrancyGuardTransient.ReentrancyGuardReentrantCall.selector` — exactly the remedy the finding requested. The guarded path is real: `src/USDRRedemption.sol:123` marks `_redeem` `nonReentrant` via the `ReentrancyGuardTransient` base (line 48), so the test genuinely exercises the guard. The client's acknowledged rationale is sound and matches the code present in this revision.

CLIENT RESPONSE · ACKNOWLEDGED

The client confirms that the reentrancy guard on `redeem` was not previously exercised by tests. They have implemented the exact remedy described in the finding: adding `test/mocks/ReentrantUSDC.sol` (a malicious mock whose transfer method re-enters `redeem` during payout) and `test_redeem_reentrancyGuarded` (which asserts the reentrant call reverts with `ReentrancyGuardReentrantCall` selector). The implementation is verifiable in commit `3633f6f` and directly exercises the `nonReentrant` guard at `src/USDRRedemption.sol:L140`.

O-04 · Inconsistent NatSpec in redeem overloads

LOCATION `src/USDRRedemption.sol:L125-L128`

Add `@dev (allowance + all-or-nothing)`, `@param usdrAmount`, and `@return usdcAmount` NatSpec to the `redeem(uint256)` overload in `USDRRedemption.sol` to match its sibling overload.

FIX VERIFICATION · ACKNOWLEDGED

The client added `src/interfaces/IUSDRRedemption.sol` with full NatSpec for `redeem(uint256)` (`@notice/@param usdrAmount/@return usdcAmount` at lines 97-100) and applied `@inheritdoc` `IUSDRRedemption` to the implementation (`src/USDRRedemption.sol:111`), so the previously-missing `@param` and `@return` docs are now present and consistent with the sibling overload. I concur with the acknowledged rationale: for an info-severity documentation finding the inheritance-based fix is sound. Minor residual nuance: the `@dev allowance/all-or-nothing` note still appears only on the two-arg overload (`USDRRedemption.sol:117-118`) and not on `redeem(uint256)`, but this is immaterial and the substantive inconsistency is addressed.

CLIENT RESPONSE · ACKNOWLEDGED

Client addressed the missing NatSpec on the `redeem(uint256)` overload by creating an `IUSDRRedemption` interface and using `@inheritdoc` to pull documentation into the implementation. This provides the required `@param usdrAmount` and `@return usdcAmount` documentation (explicitly mentioned) along with `@dev` semantics (implied by 'full NatSpec' claim) to match the sibling overload. The fix uses inheritance rather than duplication to maintain consistency.

O-05 · Add @param/@return tags to external/public surface

LOCATION src/USDRRedemption.sol:L159-L191

Add @param/@return NatSpec to redeem(uint256), fund, sweep, rescueERC20, previewRedeem, availableUSDC, maxRedeemableUSDR, sweepUnlockTime in USDRRedemption.sol to match the documented redeem(uint256,address) overload.

FIX VERIFICATION · RESOLVED

The IUSDRRedemption interface (src/interfaces/IUSDRRedemption.sol:97-145) now documents redeem(uint256), redeem(uint256,address), fund, sweep, rescueERC20, previewRedeem, availableUSDC, maxRedeemableUSDR, and sweepUnlockTime with full @param/@return tags, and USDRRedemption.sol (lines 111-225) pulls these in via @inheritdoc on every public/external member. @inheritdoc is the standard Solidity mechanism for propagating interface NatSpec to the implementation, so tooling resolves the documentation as intended. I concur with the client's 'acknowledged' rationale — the approach achieves the finding's goal of complete documentation across the external surface.

CLIENT RESPONSE · ACKNOWLEDGED

The client has addressed the NatSpec documentation finding by adding @param/@return tags across the affected public functions: redeem overloads, fund, sweep, rescueERC20, previewRedeem, availableUSDC, maxRedeemableUSDR, and sweepUnlockTime. The implementation adds documentation to the IUSDRRedemption interface and uses @inheritdoc for the public surface in USDRRedemption.sol, available in commit c9a8102. The approach is valid and achieves the intent of the finding—complete documentation visibility on the external API. The auditor should verify that the @inheritdoc inheritance produces the expected documentation visibility in tooling.

O-06 · Add IUSDRRedemption interface for integrators

LOCATION `src/USDRRedemption.sol:L43`

Consider extracting the public API into `src/interfaces/IUSDRRedemption.sol` and declaring `USDRRedemption` is `IUSDRRedemption` for integrator ergonomics (no security impact).

FIX VERIFICATION · RESOLVED

`src/interfaces/IUSDRRedemption.sol` now declares the complete public API (events `Redeemed/Funded/Swept/Rescued/Deployed`, all errors, config immutables, `redeem/fund/sweep/rescueERC20`, and views), and `src/USDRRedemption.sol:48` declares contract `USDRRedemption` is `Ownable2Step`, `ReentrancyGuardTransient`, `IUSDRRedemption` with `override` and `@inheritdoc IUSDRRedemption` on all implementing members (lines 55-220). The informational recommendation is fully implemented; I concur with the client's acknowledged rationale.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and has implemented the requested improvement. They extracted the public API, events, and errors into a new interface file (`src/interfaces/IUSDRRedemption.sol`) and updated `USDRRedemption` to declare inheritance from that interface. The implementation includes `override` and `@inheritdoc` annotations on all implementing members, following best practices. The change is present in commit `c9a8102` and is ready for auditor verification.

O-07 · Paused-USDR test uses mock revert, not fork

LOCATION test/USDRRedemption.t.sol:L225-L233

In `test_redeem_revertsWhenUSDRPaused`, relax `vm.expectRevert("Pausable: paused")` to bare `vm.expectRevert()` and add a fork test exercising the paused-USDR burn branch.

FIX VERIFICATION · RESOLVED

Both parts of the recommendation are implemented. In `test/USDRRedemption.t.sol:254` the mock assertion is relaxed from `vm.expectRevert("Pausable: paused")` to bare `vm.expectRevert()`, and `test/fork/USDRRedemption.fork.t.sol:196-215` adds `test_fork_redeem_revertsWhenUSDRPaused` which forces the live USDR token into the paused state via `stdstore` (since no external `pause()` entrypoint exists), asserts `redeem` reverts on the `whenNotPaused` burn branch, and verifies all-or-nothing rollback (`totalSupply` unchanged, receiver paid 0, `availableUSDC == FUNDING`). The test runs under both `NativeUSDC` and `USDCe` fork variants. The client's acknowledged rationale is sound: the `stdstore-on-getter-slot` approach is a legitimate workaround for the missing pause role, and the inability to execute locally is solely due to the pinned `FORK_BLOCK` aging out of public RPC retention, not a defect in the test logic.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the original finding—that `test_redeem_revertsWhenUSDRPaused` relied only on a mock revert string and lacked fork coverage (L225). They have implemented `test_fork_redeem_revertsWhenUSDRPaused` in commit 370b48e, running in the fork suite for both native USDC and USDC.e variants. The test forces USDR into paused state via `stdstore` manipulation and asserts the `redeem` reverts with the burn rolled back and no payout. The client explains a key technical constraint: the deployed USDR token exposes no external `pause()` entrypoint (all pause functions live in other contracts), so `vm.prank`-based role impersonation is infeasible; `stdstore` directly sets the paused slot (located via the getter), requiring no role or archive RPC access. The test is compile-verified but was not executed locally because the pinned `FORK_BLOCK` (88,250,000) has aged out of public Polygon RPC retention; execution requires either an archive `POLYGON_RPC_URL` endpoint or re-pinning `FORK_BLOCK` to a recent height and refreshing holder/funding fixtures.

O-08 · Bare `vm.expectRevert()` can pass on wrong error

LOCATION `test/fork/USDRRedemption.fork.t.sol:L99-L103`

In `test_fork_redeem_revertsWithoutApproval` (`USDRRedemption.fork.t.sol:101`), replace the bare `vm.expectRevert()` with the specific `ERC20InsufficientAllowance` selector and assert holder balance/funding preconditions.

FIX VERIFICATION · MITIGATED

`test/fork/USDRRedemption.fork.t.sol:116-118` adds zero-allowance, sufficient-balance, and present-funding precondition assertions before the bare `vm.expectRevert()` at line 121. These constrain `redeem`'s reachable revert paths to exactly the allowance-spend in `usdr.burn` (rate math is non-reverting, payout for 1,000 USDR is non-zero so `ZeroPayout` cannot fire, and `InsufficientUSDC` is excluded by the funding assertion), so the test can no longer pass on the wrong revert. This is a different approach than the recommended selector hardcoding, but I concur with the client's rationale that the token-version-specific reason cannot be verified offline at the pinned RPC block, and the preconditions remove the original ambiguity at its root.

CLIENT RESPONSE · ACKNOWLEDGED

Client addressed the bare `vm.expectRevert()` vulnerability by adding precondition assertions (holder balance, zero allowance, and present funding) rather than hardcoding the error selector. This approach narrows the error space so that revert can only occur from the missing-approval path. The specific error selector was deliberately not hardcoded because it cannot be verified offline at the pinned RPC block. The auditor should verify commit 8b1248a to confirm the preconditions are sufficient to eliminate ambiguity about which revert path fired.

O-09 · Add handler-based invariant tests for I1-I5

LOCATION test/USDRRedemption.t.sol:L279-L299

Add a Foundry handler-based invariant suite for USDRRedemption (ghost-tracked `sum(funded)+donated-paidOut-swept == availableUSDC()`, monotonic `lastFundingTime`, sweep locked until `lastFundingTime + 180 days`) to catch state-transition regressions.

FIX VERIFICATION · RESOLVED

test/invariant/USDRRedemption.invariant.t.sol (commit d8948cf) adds a handler-based suite that exactly implements the requested invariants: `invariant_valueConservation` asserts `totalFunded+totalDonated == totalPaidOut+totalSwept+availableUSDC()` (I1/I2 conservation, lines 116-121), `invariant_fundingTimeMonotonic` enforces monotonic and non-future `lastFundingTime` (lines 124-127, plus the in-handler `assertGe` at line 46), and `invariant_sweepUnlockTracksFunding` asserts `sweepUnlockTime()==lastFundingTime+180 days` (I5, lines 130-132) with `sweepEarlyMustRevert` exercising the `SweepLocked` timelock. The handler drives `fund/redeem/donate/sweep/warp` with ghost tracking under the configured 64 runs × 128 depth, and `forge test` confirms all 3 invariants pass with 8192 calls each and every handler selector meaningfully exercised (0 discards/reverts), so the requested coverage is genuinely present rather than vacuous. The client's acknowledged rationale is sound and the recommendation is fully implemented.

CLIENT RESPONSE · ACKNOWLEDGED

The client has implemented handler-based invariant tests for USDRRedemption in test/invariant/USDRRedemption.invariant.t.sol (commit d8948cf). The test suite includes ghost-tracked invariants covering value conservation (`funded+donated == paidOut+swept+availableUSDC()`), monotonic `lastFundingTime`, sweep-unlock timing (`sweepUnlockTime == lastFundingTime+180d`), and early-sweep reversion. The handler exercises `fund`, `redeem`, `donate`, `sweep`, and `warp` operations with 64 runs at depth 128. The auditor should verify that the implemented invariants correspond to the spec's I1-I5 and assess whether the configured bounds provide sufficient property coverage.

O-10 · Loose assertion in maxRedeemable test masks rounding

LOCATION test/fork/USDRRedemption.fork.t.sol:L138-L140

Tighten the leftover bound in `test_fork_maxRedeemable_fitsAndRedeems` (fork test L139) from `assertLt(redemption.availableUSDC(), RATE / 1e3 + 1)` to `assertLe(redemption.availableUSDC(), 1)` or assert exact residual via `previewRedeem(maxUsdr)`.

FIX VERIFICATION · RESOLVED

At test/fork/USDRRedemption.fork.t.sol:162-163 the loose bound `assertLt(availableUSDC(), RATE/1e3 + 1)` has been replaced with `assertEq(redemption.availableUSDC(), availBefore - payout)` (exact residual) plus `assertLe(redemption.availableUSDC(), 1)` (≤ 1 raw USDC unit), precisely implementing the finding's recommendation. The client's acknowledged rationale is sound and matches the code: the tightened assertions would now catch any large rounding regression the previous bound masked.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the loose assertion in `test_fork_maxRedeemable_fitsAndRedeems` was problematic and has implemented the recommended fix in commit 8b1248a. The test now asserts the exact residual via `availableUSDC == availBefore - payout` and tightens the leftover bound to ≤ 1 raw USDC unit, directly addressing the finding's concern that the previous loose bound (`RATE / 1e3 + 1`) could mask large rounding regressions.

O-11 · rescueERC20 sends strays to owner, not sender

LOCATION src/USDRRedemption.sol:L185-L191

Document that tokens (incl. mistakenly-sent USDR) transferred directly to the contract are recoverable only at the owner's discretion via rescueERC20, and steer users to the approve+redeem flow.

FIX VERIFICATION · RESOLVED

The recommended documentation has been added directly in code: src/USDRRedemption.sol:174-179 NatSpec now explicitly states the rescued balance 'goes to to (chosen by the owner, not necessarily the original sender)' and notes that any USDR balance is itself a stray transfer that is recoverable, while USDC is excluded (line 182, CannotRescueUSDC). This satisfies the finding's core requirement to document the owner-discretionary recovery mechanism; I concur with the client's acknowledged rationale, and deferring the approve+redeem user-guidance to off-chain UI is reasonable for an info-severity documentation finding.

CLIENT RESPONSE · ACKNOWLEDGED

The client agrees the issue is valid and has addressed it by updating the rescueERC20 NatSpec documentation to clearly state that recovered tokens are sent to an owner-chosen address rather than the original sender. The commit reference (c9a8102) provides a specific point for the auditor to verify the documentation change. The client notes that user guidance to follow the approve+redeem flow will be communicated through off-chain UI, which is a reasonable approach for handling that secondary recommendation. The finding's core requirement—documentation clarifying the owner-discretionary recovery mechanism—has been implemented.

O-12 · All-or-nothing redemption enables frontrun griefing

LOCATION `src/USDRRedemption.sol:L144-L145`

Document that `_redeem` is intentionally all-or-nothing FCFS (no fairness guarantee under contention) and advise large redeemers to size below `maxRedeemableUSDR()` and use an MEV-protected relay.

FIX VERIFICATION · ACKNOWLEDGED

The all-or-nothing FCFS behavior is explicitly documented in the contract NatSpec header (`src/USDRRedemption.sol:15, 21-22`: 'first-come-first-served' and 'If the contract's USDC balance cannot cover the full payout, the call reverts — no queue, no IOUs') and at the redeem overload (`L118`: 'All-or-nothing: reverts unless the full payout can be made'). The original finding was a documentation/operator-guidance recommendation for an inherent, intentional property of atomic FCFS redemption; the contract-side documentation the auditor asked for already exists, and the residual MEV/griefing risk is genuinely low and properly deferred to off-chain integrator/frontend guidance. The client's by-design rationale is sound on review.

CLIENT RESPONSE · BY DESIGN

The client confirms that the all-or-nothing FCFS behavior enabling front-running griefing is intentional and already documented in the contract header at `L144`. This is a deliberate design choice, not an oversight. The auditor's recommendation to document the behavior is already satisfied; no contract documentation changes are needed. Guidance for large redeemers to size below `maxRedeemableUSDR()` and use an MEV-protected relay is characterized as off-chain UI/integration guidance rather than a contract concern. The client accepts the residual griefing risk as inherent to the FCFS design and defers operator education to integrators and frontends.

O-13 · Redemptions depend on USDR pause state and proxy upgrades

LOCATION `src/USDRRedemption.sol:L149-L150`

Surface a clear UI/integrator error when `usdr.burn` reverts due to USDR being paused, and monitor USDR pause state and token-proxy upgrades operationally.

FIX VERIFICATION · ACKNOWLEDGED

The client's rationale is sound and verified: IUSDR.sol:10-11 explicitly documents in NatSpec that USDR's burn is whenNotPaused and that 'if Tangible pauses USDR, redemptions revert until it is unpaused (external dependency, spec |4)'. The redemption path at `src/USDRRedemption.sol:132` calls `usdr.burn` directly, so the dependency is real but the original recommendation (UI/integrator error surfacing and operational monitoring of pause state and proxy upgrades) is inherently off-chain work that cannot be addressed by contract-level code. The on-chain dependency is appropriately documented, so I concur with the client's acknowledged position for this info-severity finding.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the external dependency on USDR not being paused and on both tokens' upgradeable proxies at `src/USDRRedemption.sol:L149`. They note that this dependency is already documented in the IUSDR NatSpec. The client states that surfacing a clear UI/integrator error when `usdr.burn` reverts due to pause and monitoring USDR pause state and token-proxy upgrades are off-chain UI and operational tasks, not contract-level code changes. Therefore, they are not modifying the redemption contract itself to address this finding.

O-14 · Immutable rate makes USDR divergence fully extractable

LOCATION `src/USDRRedemption.sol:L112-L119`

Gate deployment of USDRRedemption behind a documented USDR recovery-value calc for `rate_`, and call `fund()` in small tranches so a mis-set immutable rate caps the at-risk USDC per period.

FIX VERIFICATION · ACKNOWLEDGED

The constructor at `src/USDRRedemption.sol:91-105` now enforces the validation the client describes: `IdenticalTokens` (line 93), `ZeroRate` (line 94), and decimal checks (`USDR=9`, `USDC=6` via `UnexpectedDecimals`, lines 97-99), which catch realistic miscalibration. The immutable rate is a genuine spec invariant (I3, line 31), and the residual risk of a valid-but-mispriced rate is operational, low-severity, and further bounded by tranche funding as stated. The by-design rationale is sound — the added on-chain guards plus operational tranche funding reasonably address the at-risk exposure, and immutability is intentional design rather than an oversight, so I concur.

CLIENT RESPONSE · BY DESIGN

The immutable rate mechanism is an intentional design per spec invariant I3, implemented as a constructor argument that will not be changed. The rate value is derived off-chain from documented USDR recovery value (net real-estate sale proceeds per USDR) and finalized by the Tangible team at deployment. To mitigate misconfiguration risk, commit 81a2e44 adds on-chain validation: rejection of identical token addresses (`IdenticalTokens` check) and token decimal enforcement (`USDR=9`, `USDC=6`, `UnexpectedDecimals` check), which catches the primary realistic miscalibration scenario. Funding exposure is further bounded operationally: the funder deploys capital in tranches to cap at-risk USDC per period. No further contract modifications to the rate mechanism are planned.

O-15 · Constructor doesn't validate usdr, usdc, or rate

LOCATION src/USDRRedemption.sol:L112-L119

Optionally add `if (usdr_ == usdc_) revert ZeroAddress();` and a sane upper bound on `rate_` in the USDRRedemption constructor (deploy-time hardening; no attack path).

FIX VERIFICATION · RESOLVED

USDRRedemption.sol:93 now reverts with `IdenticalTokens` when `usdr_ == usdc_`, implementing the distinct-token check from the finding. The deliberate omission of a rate upper bound is sound: there is no objective threshold for a legitimate rate, and the decimals self-check at lines 97-99 (`revert UnexpectedDecimals` unless `usdr=9/usdc=6` decimals) already fails-fast on the realistic token-misconfiguration vector that would actually misprice payouts, so an arbitrary magic-number cap would add false-rejection risk without closing a real gap. I concur with the client's rationale on the rate bound; the info-level hardening is effectively addressed.

CLIENT RESPONSE · ACKNOWLEDGED

Client implemented the `usdr != usdc` validation in commit 81a2e44, which reverts with `IdenticalTokens`. For the rate upper bound, they deliberately chose not to implement it, reasoning that no objective threshold exists and an arbitrary magic-number cap could incorrectly reject legitimate high rates. They cite the decimals self-check (finding O-17) as already guarding the realistic misconfiguration vector. Auditor should verify commit 81a2e44 and evaluate whether O-17 is indeed sufficient to prevent rate-related misconfiguration.

O-16 · Emit deployment event capturing immutable parameters

LOCATION `src/USDRRedemption.sol:L112-L119`

Add a one-time event `Deployed(address indexed usdr, address indexed usdc, uint256 rate, address indexed owner)` emitted in the constructor of `USDRRedemption.sol` for indexer/monitoring visibility (observability only, no attack path).

FIX VERIFICATION · RESOLVED

The event `Deployed(address indexed usdr, address indexed usdc, uint256 rate, address indexed owner)` is declared in `src/interfaces/IUSDRRedemption.sol:45` with all three address parameters marked indexed, exactly as the finding specified, and is emitted once in the constructor at `src/USDRRedemption.sol:104` with the immutable deployment parameters (`usdr_`, `usdc_`, `rate_`, `owner_`). The client's acknowledged rationale is sound and matches the code; the fix fully addresses the observability finding.

CLIENT RESPONSE · ACKNOWLEDGED

The client has implemented the finding by adding an event `Deployed(usdr, usdc, rate, owner)` emitted once in the `USDRRedemption` constructor, matching the audit's requirement for indexer/monitoring visibility of immutable deployment parameters. They cite commit 81a2e44 as proof, which the auditor can verify to confirm the event signature matches the specification (particularly whether address parameters are marked indexed as described in the finding).

O-17 · Reconcile configurable addresses with hardcoded decimals

LOCATION src/USDRRedemption.sol:L50-L119

Add `require(IERC20Metadata(usdr_).decimals() == 9 && IERC20Metadata(usdc_).decimals() == 6)` in the constructor to self-check the decimal assumptions baked into `USDR_UNIT` and `rate`.

FIX VERIFICATION · RESOLVED

src/USDRRedemption.sol:97-99 now reverts with `UnexpectedDecimals()` if `IERC20Metadata(usdr_).decimals() != 9 || IERC20Metadata(usdc_).decimals() != 6`, implemented in the constructor before token addresses are stored. This directly reconciles the configurable token addresses with the hardcoded `USDR_UNIT` (1e9) and 6-decimal rate assumptions, and is covered by a test (test/USDRRedemption.t.sol:98). The client's acknowledged rationale is sound and matches the code.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and confirms it has been remediated in commit 81a2e44. The constructor now includes a check that reverts with `UnexpectedDecimals` if `IERC20Metadata(usdr).decimals() != 9` or `IERC20Metadata(usdc).decimals() != 6`, directly implementing the self-check requirement to reconcile configurable token addresses with the hardcoded decimal assumptions in `USDR_UNIT` and the rate calculation.

O-18 · Event @param documentation missing

LOCATION src/USDRRedemption.sol:L79-L89

Add `@param NatSpec` to the four events (Redeemed, Funded, Swept, Rescued) in `USDRRedemption.sol` documenting indexed fields and token decimal units.

FIX VERIFICATION · RESOLVED

In `src/interfaces/IUSDRRedemption.sol:16-45` all four events (Redeemed, Funded, Swept, Rescued) plus an added `Deployed` event now carry full `@param NatSpec` documenting each field — indexed fields and decimal units are explicit (e.g. 'USDR burned, in 9-decimal raw units', 'USDC paid out, in 6-decimal raw units'). The client's acknowledged rationale is sound and matches the code: the documentation gap the info-level finding identified is fully closed.

CLIENT RESPONSE · ACKNOWLEDGED

Client acknowledges the finding and reports that `@param NatSpec` documentation has been added to all four events mentioned in the finding (Redeemed, Funded, Swept, Rescued) plus an additional `Deployed` event in the `IUSDRRedemption` interface. The fix documents indexed fields and token decimal units as required. The specific commit (c9a8102) is provided for verification.

O-19 · Funded event omits sweep-unlock time for I5 observability

LOCATION `src/USDRRedemption.sol:L162-L167`

Consider adding `uint256 sweepUnlockTime (block.timestamp + SWEEP_DELAY)` to the Funded event in `fund()` for easier indexer observability (cosmetic; value is already derivable on-chain).

FIX VERIFICATION · RESOLVED

The Funded event now carries a third field `sweepUnlockTime` (`src/interfaces/IUSDRRedemption.sol:27`), and `fund()` computes `unlockTime = block.timestamp + SWEEP_DELAY` and emits it via `emit Funded(msg.sender, usdcAmount, unlockTime)` (`src/USDRRedemption.sol:150-153`). This is exactly the recommended cosmetic enhancement; the client's acknowledged rationale is sound and matches the implemented code.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and has implemented the suggested fix by adding a `sweepUnlockTime` field to the Funded event in commit 2c7863b. The field carries the value `block.timestamp + SWEEP_DELAY`, exactly as proposed, improving off-chain observability of the I5 timelock reset sweep duration. The auditor should verify the implementation in the referenced commit to confirm the field is correctly emitted and indexed.

O-20 · Add @param/@return to view functions with only @notice

LOCATION `src/USDRRedemption.sol:L197-L213`

Add `@param/@return NatSpec` tags to `previewRedeem`, `maxRedeemableUSDR`, `availableUSDC`, and `sweepUnlockTime` in `USDRRedemption.sol` (docs consistency).

FIX VERIFICATION · RESOLVED

In `src/interfaces/IUSDRRedemption.sol`, all four view functions now carry the missing tags: `previewRedeem` (lines 130-133, `@param usdrAmount + @return`), `availableUSDC` (135-137, `@return`), `maxRedeemableUSDR` (139-141, `@return`), and `sweepUnlockTime` (143-145, `@return`). The documentation-consistency issue is fully addressed via the interface `NatSpec`, and the client's acknowledged rationale is sound and matches the code.

CLIENT RESPONSE · ACKNOWLEDGED

Client states the documentation inconsistency has been resolved in commit c9a8102 by adding `@param/@return NatSpec` tags to `previewRedeem`, `maxRedeemableUSDR`, `availableUSDC`, and `sweepUnlockTime` via the `IUSDRRedemption` interface. The client has provided sufficient detail for the auditor to verify the fix by examining the specified commit.

O-21 · Spec invariant I4 (O(1) user gas) is untested

LOCATION `src/USDRRedemption.sol:L140-L153`

Add a gas-regression test asserting redeem cost after many fund/redeem cycles matches the first call (e.g. `assertApproxEqAbs` or `vm.snapshotGas`) to guard invariant I4.

FIX VERIFICATION · RESOLVED

`test/USDRRedemption.t.sol:649-677` adds `test_redeem_gasConstantAcrossHistory`, which warms storage, captures a baseline redeem cost (L659-661), runs 50 fund/redeem cycles (L663-668), then measures a later redeem and asserts `assertApproxEqAbs(laterCost, baseline, 200)` (L676). The baseline and later measurements are symmetric (both preceded by a `_fund`), so the test genuinely guards invariant I4's O(1) user-gas property. The client's acknowledged position is sound — they implemented the requested gas-regression test as described.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and has implemented the requested fix. They added `test_redeem_gasConstantAcrossHistory` in commit 6708819, which uses `assertApproxEqAbs` with a 200 gas tolerance to assert that a redeem call after 50+ fund/redeem cycles costs the same as a warmed baseline. This directly addresses the request to add a gas-regression test guarding the O(1) user-gas invariant I4 specified at L140 of `src/USDRRedemption.sol`.

O-22 · Reconcile "stateless" claim with mutable lastFundingTime

LOCATION src/USDRRedemption.sol:L13

Qualify the @notice descriptor in USDRRedemption.sol (e.g. "no per-user state" instead of "stateless") to match invariant I4 and the mutable lastFundingTime.

FIX VERIFICATION · RESOLVED

src/USDRRedemption.sol:15-16 now reads '...USDC swap with no per-user state (the only mutable storage is the global lastFundingTime sweep clock)', replacing the imprecise 'stateless' descriptor. This qualifies the @notice to match invariant I4 and the mutable lastFundingTime storage exactly as the finding recommended; the client's acknowledged rationale is sound and the documentation change is present in the new revision.

CLIENT RESPONSE · ACKNOWLEDGED

The client recognized that the @notice descriptor 'stateless' was imprecise given the contract's mutable lastFundingTime storage. They've revised the descriptor to 'no per-user state (the only mutable storage is the global lastFundingTime sweep clock)', which accurately reflects the contract's invariant I4 and reconciles the intended design (per-user statelessness, with only a global sweep clock) with the actual storage layout. The fix directly addresses the finding's request to qualify the descriptor rather than claim full statelessness.

O-23 · Document error declarations consistently

LOCATION src/USDRRedemption.sol:L91-L101

Add one-line @dev comments to ZeroAddress, ZeroAmount, and ZeroRate in USDRRedemption.sol to match the other custom errors.

FIX VERIFICATION · RESOLVED

In the new revision the three errors were moved to src/interfaces/IUSDRRedemption.sol and each now carries a one-line @dev comment (ZeroAddress L51-52, ZeroAmount L53-54, ZeroRate L55-56), matching the documentation style of the other custom errors. The client's acknowledged rationale is sound and confirmed by direct inspection.

CLIENT RESPONSE · ACKNOWLEDGED

Client acknowledges the finding and states the issue has been resolved in commit c9a8102 by adding @dev NatSpec documentation to the three errors (ZeroAddress, ZeroAmount, ZeroRate) in the IUSDRRedemption interface to match the documentation style of other custom errors. The auditor can verify the fix by inspecting the referenced commit.

O-24 · ZeroPayout NatSpec is misleading when usdrAmount is zero

LOCATION `src/USDRRedemption.sol:L140-L142`

Add `if (usdrAmount == 0) revert ZeroAmount();` at the top of `_redeem` in `USDRRedemption.sol` so zero input maps to the same error as `fund`, or broaden the `ZeroPayout @dev NatSpec` to cover the zero-amount case.

FIX VERIFICATION · ACKNOWLEDGED

The `ZeroPayout` error `NatSpec` at `src/interfaces/IUSDRRedemption.sol:61-62` was broadened to 'The redemption would pay out zero USDC — the amount (zero included) is too small for the rate to yield a non-zero payout,' which now explicitly covers the zero-amount case. This is exactly one of the two remediation options the finding offered, and `_redeem` (`src/USDRRedemption.sol:124-125`) was intentionally left unchanged to preserve existing test behaviour. The client's acknowledged rationale is sound: for an informational documentation issue, clarifying the `NatSpec` fully resolves the misleading-error concern without altering contract logic.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the misleading `NatSpec` in the original `ZeroPayout` error and has resolved it by broadening the `@dev` comment to clarify that amounts too small to yield a non-zero payout—including zero—revert with this error. They chose this approach over adding a separate `ZeroAmount` check in `_redeem` to preserve consistent behaviour and avoid modifying existing tests. The fix is grounded in commit `c9a8102` and respects one of the two options suggested in the finding.

O-25 · Sweep timelock transfers value from passive to active holders

LOCATION `src/USDRRedemption.sol:L172-L179`

Optional UX hardening for `sweep()` in `USDRRedemption.sol`: emit the unlock schedule / add a public grace-period event before the 180-day `SWEEP_DELAY` window so passive USDR holders are warned (design tradeoff, no current defect).

FIX VERIFICATION · ACKNOWLEDGED

The original info-level finding asked for optional UX hardening — emitting the sweep unlock schedule so passive holders can monitor the wind-down clock. The new code does exactly this: `fund()` now computes and emits the unlock timestamp via `Funded(funder, usdcAmount, unlockTime)` (`src/USDRRedemption.sol:149-153`, event declared at `src/interfaces/IUSDRRedemption.sol:26-27`), and a public `sweepUnlockTime()` view exposes `lastFundingTime + SWEEP_DELAY` on demand (`src/USDRRedemption.sol:220-225`). I concur with the client's rationale: per-funding unlock visibility plus an always-queryable view is sufficient warning for an info-severity design tradeoff, and declining a separate pre-window grace-period event is reasonable since the timelock value-transfer is accepted as intentional with no underlying defect.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the wind-down deadline game and has addressed the observability concern by modifying the `Funded` event to emit `sweepUnlockTime` (commit 2c7863b), allowing passive holders to monitor the unlock schedule before the 180-day `SWEEP_DELAY` window. They explicitly decline to add a separate pre-window grace-period event, contending that per-funding unlock timestamp visibility is sufficient warning. The timelock value-transfer tradeoff is accepted as intentional, with added transparency as the primary mitigation.

O-26 · Fuzz tests never vary receiver or two-arg redeem

LOCATION test/USDRRedemption.t.sol:L279-L299

Add a fuzz test varying receiver through the two-arg redeem(uint256,address) overload, asserting payout lands at the resolved receiver (address(0) → msg.sender).

FIX VERIFICATION · RESOLVED

test/USDRRedemption.t.sol:608-623 adds testFuzz_redeem_receiverRouting, which fuzzes the receiver through the two-arg redeem(uint256,address) overload and asserts payout lands at the resolved receiver, with address(0) mapping to the redeemer (line 620: receiver == address(0) ? alice : receiver) and a check that the redeemer is not paid when distinct (line 622). This matches the contract's resolution logic in src/USDRRedemption.sol:120 (receiver == address(0) ? msg.sender : receiver), so the recommended test coverage is now present and correct.

CLIENT RESPONSE · ACKNOWLEDGED

The client has addressed the finding by implementing the requested test. They added testFuzz_redeem_receiverRouting, which varies the receiver parameter through the two-argument redeem(uint256,address) overload and asserts that payout lands at the resolved receiver, with address(0) correctly mapping to msg.sender. The client provides sufficient detail for the auditor to verify the implementation matches the finding's requirements (commit 6708819).

O-27 · Rate math only tested at single value (541_700)

LOCATION test/USDRRedemption.t.sol:L107-L112

Parameterize the math fuzz tests (testFuzz_previewRedeem_isFloorOfExactProduct, testFuzz_redeem_conservation) over rate_ via bound(rate_, 1, 1e12) and a new USDRRedemption per run, with amount = bound(amount, 0, type(uint256).max / rate_) to cover the rate = 1 ZeroPayout boundary and overflow paths.

FIX VERIFICATION · RESOLVED

test/USDRRedemption.t.sol:626-636 adds testFuzz_previewRedeem_parameterizedRate, which bounds rate_ to [1, 1e12], deploys a fresh USDRRedemption per run, bounds amount to [0, type(uint256).max / rate_] to avoid the product overflow, asserts the floor formula payout == (amount*rate_)/1e9, and explicitly exercises the rate==1 ZeroPayout boundary (amount < 1e9 → payout 0). This implements every requirement in the original recommendation. The client's acknowledged rationale is sound and matches the code present at the commit.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding's validity and confirms implementation of parameterized rate-math fuzz testing via commit 6708819. A new testFuzz_previewRedeem_parameterizedRate test bounds rate_ to [1, 1e12] and amount to [0, type(uint256).max / rate_] to prevent overflow, deploys a fresh USDRRedemption instance per test run, and asserts the correct payoff formula (payout == amount*rate_/1e9). The implementation explicitly covers the rate=1 ZeroPayout boundary case (amount < 1e9 → payout 0), directly addressing all requirements stated in the original finding.

O-28 · Fork never tests USDC pause or blacklist path

LOCATION `test/fork/USDRRedemption.fork.t.sol:L61-L97`

Add a native-USDC fork test driving the blacklist/pause path (e.g. `test_fork_redeem_revertsWhenReceiverBlacklisted` in `test/fork/USDRRedemption.fork.t.sol`) to guard the USDC-side external-failure path.

FIX VERIFICATION · RESOLVED

The recommended test `test_fork_redeem_revertsWhenReceiverBlacklisted` now exists in `test/fork/USDRRedemption.fork.t.sol:169-188`: it impersonates the live `FiatToken blacklist()` (interface at lines 18-21), blacklists the receiver, calls `redemption.redeem(redeemAmount, receiver)` under `vm.expectRevert()`, and asserts the USDR burn rolled back (`usdr.totalSupply() == supplyBefore`, line 186) and funding is untouched (line 187). Because the test lives in the abstract `USDRRedemptionForkTestBase` and the recommendation's path is exercised via `_usdcToken()`, both concrete suites — `USDRRedemptionForkTest_NativeUSDC` (line 241) and `USDRRedemptionForkTest_USDCe` (line 247) — run it, so the native-USDC and bridged USDC.e external-failure paths are both covered. I concur with the client's rationale: the recommended coverage is implemented in code, and the inability to execute at runtime is a pre-existing pruned-RPC limitation affecting the entire fork suite, orthogonal to this finding.

CLIENT RESPONSE · ACKNOWLEDGED

The client implemented the requested fork test `test_fork_redeem_revertsWhenReceiverBlacklisted` in commit 8b1248a, which exercises the USDC-side external-failure path by impersonating the `FiatToken blacklist()`, blacklisting the receiver, and verifying the redeem reverts with USDR burn rolled back. The test covers both native USDC and bridged USDC.e variants. Compile-time verification via `forge build` has passed. Runtime execution validation is deferred due to infrastructure constraints: all public Polygon RPC endpoints are pruned and cannot serve archived state at block 88,250,000, which is a pre-existing condition affecting the entire fork suite. To complete validation, the fork suite must be run against an archive node via the `POLYGON_RPC_URL` environment variable. No pending code changes remain.

O-29 · Tautological assertion in test_views_capacity

LOCATION test/USDRRedemption.t.sol:L501-L502

In test_views_capacity (USDRRedemption.t.sol:502), replace the formula-recomputed expression with an independently-derived literal: `assertEq(redemption.maxRedeemableUSDR(), 1_000_001_846)` (not the finding's incorrect `1_000_000_001`).

FIX VERIFICATION · RESOLVED

test/USDRRedemption.t.sol:571 now asserts the independently-derived literal `assertEq(redemption.maxRedeemableUSDR(), 1_000_001_846)`, with a preceding comment (L568-569) deriving the value as `floor(541_701 * 1e9 / 541_700)` rather than reusing the contract's formula. This exactly matches the recommendation and the client's stated fix, so the assertion is now meaningful and non-tautological. I concur with the client's acknowledged rationale.

CLIENT RESPONSE · ACKNOWLEDGED

The client has addressed the tautological assertion in test_views_capacity (USDRRedemption.t.sol:L501) by replacing the formula-recomputation with an independently-derived literal value (`1_000_001_846`), as recommended in the finding. The fix has been implemented in commit 6708819. This change ensures the test validates the contract's maxRedeemableUSDR view against an independent calculation rather than reproducing the contract's own logic, making the assertion meaningful and non-redundant.

O-30 · Zero-balance paths in sweep and rescueERC20 untested

LOCATION `src/USDRRedemption.sol:L172-L191`

Add zero-balance regression tests for sweep and rescueERC20 asserting `Swept(to, 0) / Rescued(token, to, 0)` succeed (test-coverage hardening; no current issue).

FIX VERIFICATION · RESOLVED

`test/USDRRedemption.t.sol:491-499` (`test_sweep_zeroBalanceSucceeds`) warps past the timelock and asserts emit `Swept(bob, 0)` succeeds with zero transfer, and `test/USDRRedemption.t.sol:546-554` (`test_rescue_zeroBalanceSucceeds`) asserts emit `Rescued(stray, bob, 0)` on a token the contract holds none of. Both use `vm.expectEmit` with full topic/data matching, directly closing the zero-balance coverage gap. The client's 'acknowledged' rationale is sound — the recommended regression tests were added in letter and spirit.

CLIENT RESPONSE · ACKNOWLEDGED

The client has directly addressed the finding by adding two regression tests: `test_sweep_zeroBalanceSucceeds` and `test_rescue_zeroBalanceSucceeds` in commit 6708819. These tests assert that the `Swept(to, 0)` and `Rescued(token, to, 0)` events succeed on zero-balance calls, which closes the test-coverage gap identified in the original finding for the sweep and rescueERC20 functions at `src/USDRRedemption.sol:L172`. The auditor can verify this by examining the referenced commit and confirming the test implementations match the specified assertions.

O-31 · Inconsistent zero-address handling in redeem vs owner

LOCATION `src/USDRRedemption.sol:L136-L186`

Add a one-line @dev note on `sweep/rescueERC20` clarifying that, unlike `redeem`, owner withdrawal targets are not zero-coerced and must be explicit.

FIX VERIFICATION · RESOLVED

Both owner withdrawal functions now carry the requested NatSpec notes: `sweep` (`src/USDRRedemption.sol:157-159`) states 'Unlike {redeem}, to is not zero-coerced — a zero address reverts rather than defaulting to `msg.sender`', and `rescueERC20` (`src/USDRRedemption.sol:178-179`) states '...like {sweep}, to is not zero-coerced.' Both functions also enforce this with `if (to == address(0)) revert ZeroAddress();` (lines 161, 181). This fully satisfies the informational finding's request to document the inconsistency, and the client's acknowledged rationale is sound.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and has implemented the requested documentation fix. They state that commit `c9a8102` adds NatSpec notes to `sweep` and `rescueERC20` clarifying that, unlike `redeem`, their to target is not zero-coerced and must be explicit, with a zero address reverting. This directly addresses the finding's request to clarify the inconsistency in zero-address handling between `redeem` and the owner withdrawal functions.

O-32 · rescueERC20 lacks reentrancy guard (owner-gated)

LOCATION `src/USDRRedemption.sol:L185-L191`

Add `nonReentrant` to `rescueERC20` (and optionally `sweep/fund`) in `USDRRedemption.sol` for consistency; guard is already imported (hardening, no current attack path).

FIX VERIFICATION · RESOLVED

In the new revision `src/USDRRedemption.sol`, `rescueERC20` now carries the `nonReentrant` modifier (line 180), and the recommendation's optional targets `fund` (line 145) and `sweep` (line 160) are guarded as well, using the EIP-1153 transient `ReentrancyGuardTransient` already imported (line 9). This matches the hardening recommendation exactly, and I concur with the client's acknowledged rationale: `rescueERC20` transfers arbitrary attacker-controlled tokens whose transfer hooks could re-enter, so the defense-in-depth guard is the meaningful and correct mitigation.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding and has implemented the fix in commit `b6ded55` by adding the `nonReentrant` modifier to `rescueERC20`, `sweep`, and `fund`. The client correctly identifies `rescueERC20` as the meaningful case, noting that it transfers arbitrary attacker-controlled tokens whose hooks could re-enter during the transfer. This hardening measure aligns with the audit recommendation for defense-in-depth, even though no active attack path was identified.

O-33 · FCFS redemptions are frontrunnable when USDC is scarce

LOCATION `src/USDRRedemption.sol:L140-L153`

Ensure front-ends surface the `maxRedeemableUSDR` caveat so users know redeem can revert with `InsufficientUSDC` when reserves are thin (intended FCFS behavior; no code change).

FIX VERIFICATION · ACKNOWLEDGED

The finding is informational and explicitly recommended no contract change — only that front-ends surface the FCFS caveat. The code at `src/USDRRedemption.sol:123-136` retains the intended all-or-nothing FCFS behavior (`if (usdcAmount > available) revert InsufficientUSDC` at L128), and the `maxRedeemableUSDR()` view (`src/USDRRedemption.sol:213`, `IUSDRRedemption.sol:141`) is available for front-ends to query the cap, exactly as the recommendation envisioned. The client's acknowledged rationale is sound: the revert-when-thin behavior is by design, the residual risk is low (no fund loss — redeemers simply retry), and committing to UI disclosure directly implements the only action the finding asked for.

CLIENT RESPONSE · ACKNOWLEDGED

The client acknowledges the finding that FCFS redemptions at `src/USDRRedemption.sol:L140` can revert with `InsufficientUSDC` when USDC reserves are thin, creating a frontrunning risk. They confirm this behavior is intentional and by design—the FCFS mechanism operates as intended—and that no contract code changes are planned. They commit to addressing user awareness through UI/front-end work by surfacing the `maxRedeemableUSDR` caveat and clearly communicating the possible `InsufficientUSDC` revert to users, which directly implements the finding's recommendation.

O-34 · Inconsistent visibility among view functions

LOCATION src/USDRRedemption.sol:L197-L218

Add a comment noting `previewRedeem` and `availableUSDC` must stay public (reused internally by `_redeem` and `maxRedeemableUSDR`) to prevent accidental downgrade to external.

FIX VERIFICATION · ACKNOWLEDGED

In the new revision, `src/USDRRedemption.sol:193` annotates `previewRedeem` with `@dev Kept public (not external): {_redeem}` and `{maxRedeemableUSDR}` call it internally. and line 199 annotates `availableUSDC` with `@dev Kept public (not external): {_redeem}, {sweep}` and `{maxRedeemableUSDR}` call it. Both functions remain public, directly implementing the finding's recommendation; the client's acknowledged rationale is sound and the annotations are accurate.

CLIENT RESPONSE · ACKNOWLEDGED

The client confirmed that `previewRedeem` and `availableUSDC` must remain public because internal functions (`_redeem`, `sweep`, and `maxRedeemableUSDR`) call them. They've added `@dev` notes to both functions documenting this visibility constraint, which directly addresses the finding's recommendation to prevent accidental downgrade to external. The implementation in commit `c9a8102` can be verified to confirm the annotations are in place and accurate.

O-35 · Use availableUSDC() helper consistently

LOCATION src/USDRRedemption.sol:L144-L176

Replace `inline usdc.balanceOf(address(this))` in `_redeem` (line 144) and `sweep` (line 176) with the `availableUSDC()` helper for consistency.

FIX VERIFICATION · RESOLVED

In the new revision, `_redeem` reads the balance via `uint256 available = availableUSDC();` (`src/USDRRedemption.sol:127`) and `sweep` via `uint256 balance = availableUSDC();` (line 169), replacing the `inline usdc.balanceOf(address(this))` reads. The only remaining direct `balanceOf(address(this))` call is in `rescueERC20` (line 183), which correctly operates on an arbitrary token rather than USDC and so should not use the helper. The client's acknowledged rationale is sound and matches the code.

CLIENT RESPONSE · ACKNOWLEDGED

The client has remediated the finding by replacing `inline usdc.balanceOf(address(this))` reads in both `_redeem` (line 144) and `sweep` (line 176) with calls to the `availableUSDC()` helper, as specified. The client cites commit `d930fd8` and notes that this establishes a single source of truth for reading the contract's USDC balance, improving consistency and maintainability.

O-36 · previewRedeem overflow boundary never fuzzed

LOCATION test/USDRRedemption.t.sol:L107-L112

Optionally add a boundary fuzz case asserting previewRedeem panics (0x11) near the $\text{usdrAmount} * \text{rate}$ overflow threshold, or document the supply ceiling makes it unreachable (test-only; no contract change).

FIX VERIFICATION · RESOLVED

test/USDRRedemption.t.sol:641-645 adds testFuzz_previewRedeem_overflowBoundary, which bounds usdrAmount to $[\text{type}(\text{uint256}).\text{max}/\text{RATE} + 1, \text{type}(\text{uint256}).\text{max}]$ and asserts previewRedeem reverts with stdError.arithmeticError (0x11). This precisely pins the multiplication-overflow threshold of previewRedeem's $\text{usdrAmount} * \text{rate} / \text{USDR_UNIT}$ (src/USDRRedemption.sol:195), since overflow of $\text{usdrAmount} * \text{rate}$ begins exactly when $\text{usdrAmount} > \text{max}/\text{rate}$. The client's acknowledged rationale is sound and the test is correctly implemented, fully addressing the optional test-only suggestion.

CLIENT RESPONSE · ACKNOWLEDGED

The client has implemented the suggested boundary fuzz test for the previewRedeem multiplication overflow case. Commit 6708819 added testFuzz_previewRedeem_overflowBoundary, which fuzzes usdrAmount values above $\text{type}(\text{uint256}).\text{max}/\text{RATE}$ and asserts that previewRedeem correctly panics with the arithmetic overflow error (0x11) rather than wrapping. This directly addresses the optional test improvement suggested in the finding.

Appendix · Methodology

The audit was carried out across several independent specialised passes — security analysis, business-logic verification, architectural review, code-quality assessment, and (where applicable) zero-knowledge circuit analysis, economic and game-theoretic review, gas-optimisation review, and test-suite analysis. Each pass examined the source independently; findings were then consolidated, deduplicated, and verified for accuracy before inclusion in this report.

This is a **fix-review (v2)** of the engagement. The initial audit was performed at `45611d4` ; the development team subsequently applied fixes, and the fix-review pass examined the affected code at `ac06719` . For each prior-version finding, the verifier read the new code at the affected location (and related call sites where relevant) and assigned one of the verdicts described in the **Fix Status Scale** appendix below. Findings discovered during a fix-review pass (rather than carried forward from the prior version) are flagged with a **NEW** badge and are subject to the same severity scale as the original findings.

Location references follow the finding's status: a resolved, mitigated, or acknowledged finding cites the line numbers from the **initial audit commit** (`45611d4`), where the issue was identified and where it no longer poses a risk; an unresolved, partially-resolved, or newly-discovered finding cites the **fix-review commit** (`ac06719`), where the issue still lives and a reader can inspect it directly.

Appendix · Severity Scale

-  **Critical** Direct, reachable theft or destruction of user funds, or permanent loss of contract availability, exploitable by an unprivileged attacker with realistic preconditions. Must be fixed before any production deployment.
-  **High** Fund loss or significant privilege escalation requiring specific preconditions, or material centralisation risk that materially undermines the trust model. Must be fixed before the next release.
-  **Medium** Limited fund loss, denial of service, or operational risk; correctness defect that does not directly enable theft but weakens the protocol's safety posture. Should be addressed in the next planned release.
-  **Low** Minor issues, hardening recommendations, defensive-coding improvements, or edge cases with limited impact. Should be tracked but do not block deployment.
-  **Info** Best-practice notes, documentation gaps, and observations with no exploitable impact.

Appendix · Fix Status Scale

-  **Resolved** The development team's fix correctly addresses the root cause. The original attack or failure scenario no longer applies at the fix-review commit.
-  **Partially Resolved** The fix addresses part of the issue but leaves residual exposure — either a narrower attack path remains open, or the fix relies on conditions that are not fully enforced.
-  **Mitigated** The underlying defect is not removed, but its impact has been reduced to an acceptable level (through compensating controls, scope limits, or operational guard-rails). Residual risk is documented in the finding.
-  **Acknowledged** The client provided a response (see the Client Response block on the finding) accepting the residual risk, declaring the behaviour by-design, or otherwise opting not to change the code; the auditor reviewed the rationale and concurs that it is sound. The finding stays in the report so the dialogue is on the record.
-  **Unresolved** The fix is missing, incorrect, or insufficient. The original finding remains exploitable at the fix-review commit and continues to require remediation before production deployment.
-  **Newly Discovered** This finding was identified during the fix-review pass rather than carried forward from the initial audit. It is subject to the same severity scale as the original findings.

Appendix · Scope

This report covers the USDR Redemption V2 codebase as observed at two commits: the initial audit at `45611d4` and the fix-review at `ac06719`. 4 findings are documented; each is identified by a per-severity display ID (`C-01`, `H-02`, `M-01`, ...) used consistently in the executive summary, the contents page, and the body. Findings newly discovered during fix-review are listed alongside the carried-forward ones and carry a **NEW** badge.

Out of scope unless explicitly noted: deployment scripts, external dependencies (OpenZeppelin, Solmate, upstream Noir / Barretenberg, etc.), frontend applications, off-chain infrastructure, and any code not referenced in a finding's Location field.

Appendix · Disclaimer

This report is provided for informational purposes and represents the auditor's professional opinion at the time of review. It is **not** a guarantee that the audited code is free of defects, exploits, or vulnerabilities. Security audits reduce risk; they do not eliminate it. The client remains solely responsible for any decision to deploy, operate, or rely on the audited code in production.

Cimora accepts no liability for losses arising from the use, deployment, or operation of the audited code. The terms of engagement are governed by the executed Audit Services Agreement.

Appendix · Related-Party Disclosure

The reviewer is not independent of the audited code. Cimora's principal is the lead developer of the USDR redemption contract reviewed here (`src/USDRRedemption.sol`), a co-founder of Tangible (the issuer of USDR), and a holder of USDR; he also operates personal trading and arbitrage bots that interact with USDR. Because the reviewer authored the contract under audit, this engagement is a self-review by the code's author, not an independent third-party security audit. Readers must not treat the findings — or the absence of findings — as an independent assessment of the code's safety, and an unaffiliated audit is recommended before relying on this contract in production.

Readers relying on this report are advised that the auditor and the audited entity are not independent in the corporate sense. This disclosure is provided so that any party making a decision based on this report can weigh it appropriately.